Reviewer Pack — Minimal Rank of Quandle Operations in Combinatorial Species …

Entry a1b666651cba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 21:52:32 UTC

Minimal Rank of Quandle Operations in Combinatorial Species vs ACC⁰ Circuit Lower Bounds for XOR-AND Networks

Entry ID: `a1b666651cba` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 21:52:32 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Species Theory **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity

Statement:

> ['For every instance of an XOR-AND network with n inputs, the minimal rank of the quandle operations in its combinatorial species representation is upper-bounded by a function f(n) = Θ(log^2(n)).', 'Equivalently, for any ACC⁰ circuit C computing an XOR-AND network, the size of C is at most f(n).', 'There exists a constructive mapping from an XOR-AND network instance to its combinatorial species representation that can be computed in polynomial time.']

Rationale (proposer's reasoning):

> ['Quandle operations in combinatorial species provide a rich algebraic structure that may capture the complexity of boolean functions at a finer level than traditional representations.', 'ACC⁰ circuits are known to be a suitable model for studying the complexity of XOR-AND networks, and their size is directly related to the difficulty of computing the network.', 'This conjecture aims to bridge these two fields by proposing a quantitative relationship that could potentially lead to new insights into the complexity of boolean functions.']

Taxonomy category: `COMBINATORIAL_SPECIES_XOR_AND_NETWORKS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `068b5c8fb95f395e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the conjecture about quandle operations in combinatorial species representation and ACC⁰ circuits, support is achieved if the minimal rank of quandle operations (r) is within f(n) = Θ(log^2(n)) for all seeds, AND the size of the ACC⁰ circuit (s) does not exceed f(n). Falsification occurs if any seed has r > Θ(log^2(n)) OR s > f(n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `combinatorial species AND quandle operations AND ACC⁰ circuit complexity` - `XOR-AND networks AND minimal rank AND combinatorial species representation` - `polynomial time mapping XOR-AND network combinatorial species`

Top relevant hits considered: - [http://arxiv.org/abs/math/0501315v2] Taming the wild in impartial combinatorial games - [http://arxiv.org/abs/math/0305031v2] Random Combinatorial structures:the convergent case - [http://arxiv.org/abs/1109.1216v1] Combinatorial representations - [http://arxiv.org/abs/1911.01153v1] Fast Reliability Ranking of Matchstick Minimal Networks - [http://arxiv.org/abs/1107.0539v1] Corporate competition: A self-organized network - [http://arxiv.org/abs/2507.01136v2] Minority representation and fairness in network ranking: An application to school contact diary data - [http://arxiv.org/abs/gr-qc/9810059v1] Space-time distributions - [http://arxiv.org/abs/2002.12771v2] Tuning as convex optimisation: a polynomial tuner for multi-parametric combinatorial samplers

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    augmented = [[A[i][j] for j in range(n)] + [b[i]] for i in range(n)]

    for i in range(n):
        # Find pivot row
        max_row = i
        for k in range(i+1, n):
            if abs(augmented[k][i]) > abs(augmented[max_row][i]):
                max_row = k

        # Swap rows
        augmented[i], augmented[max_row] = augmented[max_row], augmented[i]

        # Eliminate below pivot
        for k in range(i+1, n):
```

```
            factor = augmented[k][i] / augmented[i][i]
            for j in range(n + 1):
                augmented[k][j] -= factor * augmented[i][j]

    # Back-substitute to find solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = augmented[i][-1]
        for j in range(i+1, n):
            x[i] -= augmented[i][j] * x[j]
        x[i] /= augmented[i][i]

    return x

def compute_quandle_rank(gates):
    A = []
    b = []
    for gate in gates:
        row = [0] * len(gate)
        row[gate[1]] = 1
        A.append(row)
        b.append(-gate[2])

    try:
        solution = gaussian_elimination(A, b)
        rank = sum(1 for x in solution if not math.isclose(x, 0))
        return rank
    except ZeroDivisionError:
        return None

def generate_xor_and_network(n):
    gates = []
    inputs = [i for i in range(n)]
    outputs = [n + i for i in range(n)]

    # Generate random XOR-AND gates
    for _ in range(2 * n):
        gate_type = random.choice(['XOR', 'AND'])
        if gate_type == 'XOR':
            inputs1, inputs2 = random.sample(inputs, 2)
            output = random.choice(outputs)
            gates.append((inputs1, inputs2, output))
        else:
            inputs1, inputs2 = random.sample(inputs, 2)
            output = random.choice(outputs)
            gates.append((inputs1, inputs2, output))

    return gates

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    gates = generate_xor_and_network(n)

    quandle_rank = compute_quandle_rank(gates)
    if quandle_rank is None:
```

```
        return {
            "metric_name": "Quandle Rank",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    log_n_squared = math.log(n, 2) ** 2
    conjecture_holds = quandle_rank <= log_n_squared

    return {
        "metric_name": "Quandle Rank",
        "metric_value": quandle_rank,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"quandle_rank={quandle_rank}, log^2(n)={log
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"quandle_rank > log^2(n)\" first_failing_seed={f
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3fd30259.py", line 115, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3fd30259.py", line 88, in run_trial
    quandle_rank = compute_quandle_rank(gates)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3fd30259.py", line 53, in compute_quand
    row[gate[1]] = 1
    ~~~^^^^^^^^^^
IndexError: list assignment index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means we cannot verify if the minimal rank of quandle operations and the size of the ACC^0 circuit are within the bounds specified by f(n) = Θ(log^2(n)). | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results for analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13840
2	propose	ollama_remote	glm4:latest	0	0	13644
3	preregistration	ollama_remote	glm4:latest	0	0	9796
4	novelty	ollama_remote	glm4:latest	0	0	8664
5	novelty	ollama_remote	glm4:latest	0	0	10064
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11667
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9238
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13250
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12609
10	judge	ollama_remote	glm4:latest	0	0	16643

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 119416 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/a1b666651cba.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a1b666651cba.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a1b666651cba.tar.gz` (if generated)